

Save Philly Festivals - Technical Guide for Developers



Audience: Developers and technical operators **Scope:** Repository architecture, application implementation, N8N integration boundary, test/release workflows **Last updated:** 2026-08-05

This guide is the technical companion to `docs/Client-UserGuide.md` . It describes how to safely build, test, and operate this repository without activating external integrations accidentally.

1. Repository architecture

This repository is an Nx + pnpm monorepo with two deployable applications:

Project	Location	Runtime	Purpose
Web application	<code>apps/save-philly-festivals</code>	Next.js 16 + Auth.js + Prisma 7 + PostgreSQL	Public discovery, producer submissions, editorial/admin workflows
Automation application	<code>apps/n8n</code>	Docker Compose (N8N + dedicated PostgreSQL + Caddy in production model)	Controlled workflows, currently inactive by default

Key root files:

- `package.json` - workspace scripts and quality gates
- `pnpm-workspace.yaml` - workspace package scope
- `nx.json` - Nx defaults and project graph behavior
- `README.md` - top-level setup and quality gate commands
- `remaining-production-gates.md` - production-readiness checklist

2. Local setup and prerequisites

2.1 Prerequisites

- Node.js 20.19+
- Corepack + pnpm 11.13.0
- PostgreSQL for web app migrations/tests
- Docker + Compose for N8N

2.2 Initial setup (workspace root)

```
corepack enable
pnpm install --frozen-lockfile
cp apps/save-philly-festivals/.env.example apps/save-philly-festivals/.env.local
pnpm run auth:secret
pnpm exec nx run save-philly-festivals:prisma-generate --outputStyle=static
pnpm exec nx run save-philly-festivals:migrate-test --outputStyle=static
pnpm run db:seed
pnpm run db:verify-admin
pnpm run dev
```

Important:

- Never commit `.env.local` or any secret-bearing env file.
- `migrate-test` expects a disposable blank PostgreSQL database path in `DATABASE_URL`.
- Production schema releases use `prisma migrate deploy`, not `prisma db push`.

3. Application architecture (apps/save-philly-festivals)

3.1 Route and feature model

- App Router routes live under `src/app`.
- Feature/domain logic is organized under `src/features` and `src/lib`.
- Prisma schema and migration history are under `prisma/`.

Public behavior is intentionally bounded:

- Only published festivals are visible publicly.
- Approval and publication are separate actions.
- Canceled festivals use a tombstone model, not deletion.

3.2 Data and persistence

- PostgreSQL is the source of truth for users, festivals, workflow state, consent/outbox records, and delivery logs.
- Browser-local schedule state is stored client-side under a versioned key and is intentionally non-account-based.
- Sensitive producer contact fields are kept private and excluded from public payloads.

3.3 Time model

- Canonical timezone is `America/New_York`.
- Date-only and timed events are handled differently by design (calendar export all-day end dates are exclusive per iCalendar).

4. N8N architecture and boundary

N8N is isolated from the web application runtime and database.

4.1 Safety boundary

- Local N8N binds to loopback (`127.0.0.1:5678`).
- Production model publishes only via HTTPS reverse proxy.
- Workflows in this repository are inactive exports by default.
- `DiasporaDNA.json` is draft-only behavior.
- `OrganizerSubscriptions.json` is an inactive contract boundary and requires explicit provider implementation before use.

4.2 Organizer subscription contract

The only approved application-to-N8N organizer-mailing boundary is via:

- `POST /api/internal/n8n/organizer-subscriptions/claim`
- `POST /api/internal/n8n/organizer-subscriptions/report`

Requirements:

- Shared bearer secret in both app and N8N secret stores (`N8N_ORGANIZER_OUTBOX_SECRET`).
- Strict lease-token reporting semantics.
- Idempotency, suppression, and retries are enforced by application domain rules.

5. Quality gates and release gates

Run from workspace root:

```
pnpm run lint
pnpm run test
pnpm run test:coverage
pnpm run build
pnpm exec playwright install chromium
pnpm run e2e
pnpm run n8n:validate
pnpm run n8n:validate:production
pnpm run n8n:test
pnpm audit
pnpm run verify
```

Merge policy summary:

- `main` is always releasable.
- No merge without required gates green on the exact commit.
- Flaky tests block merge; retries can gather diagnostics but do not establish pass.
- Merging integration code does not authorize activation of providers.

See `docs/DELIVERY-WORKFLOW.md` for full policy.

6. N8N local operations (developer path)

```
pnpm run n8n:init
pnpm run n8n:validate
pnpm run n8n:test
pnpm run n8n:plan
CONFIRM_DEPLOY=n8n pnpm run n8n:deploy
pnpm run n8n:health
pnpm run n8n:logs
pnpm run n8n:stop
```

Operational notes:

- `n8n:init` creates ignored local env files and does not overwrite existing files.
- Do not use destructive volume deletion as a normal reset path.
- Validate both local and production Compose models before any operational proposal.

7. CI/CD and deployment model

- CI workflow (`.github/workflows/ci-cd.yml`) enforces migration, lint, tests, build, and E2E.
- N8N DigitalOcean operations are separated into a dedicated guarded workflow.
- Production N8N deployment requires protected environment approval and confirmation string.
- Workflow import and workflow activation are separate approvals.

Required N8N DigitalOcean secrets are listed in root `README.md` and `apps/n8n/README.md`.

8. Security and data handling rules

- Parameterize all database queries.
- Keep provider credentials in secret managers only.
- Never paste credentials/tokens into workflow JSON, docs, or tickets.
- Do not log unnecessary PII in production behavior.
- Preserve immutable audit trails for workflow and moderation decisions.

9. Common developer tasks

9.1 Add a web feature safely

1. Branch from up-to-date `main`.
2. Add unit/integration tests first or with implementation.
3. Add/adjust Playwright coverage when user-visible behavior changes.
4. Run required gates.

5. Open PR with evidence.

9.2 Add or change schema

1. Update `schema.prisma`.
2. Create migration and review SQL.
3. Verify blank-database migration via `migrate-test`.
4. Include rollback/forward notes in PR.

9.3 Change N8N workflow safely

1. Keep workflow inactive in repository export.
2. Update static/fixture tests.
3. Validate local + production Compose configs.
4. Document any new credentials/scopes required.
5. Do not claim provider compatibility without controlled proof.

10. Related documentation

- `README.md`
- `apps/save-philly-festivals/README.md`
- `apps/n8n/README.md`
- `docs/Client-UserGuide.md`
- `docs/DELIVERY-WORKFLOW.md`
- `docs/FDE-DELIVERY-PLAN.md`
- `docs/SCHEDULE-CALENDAR-EMAIL.md`
- `docs/PRODUCER-SUBMISSION-OPERATIONS.md`
- `docs/SOCIAL-FEED-OPERATIONS.md`
- `docs/FESTIVAL-DATA-IMPORT-RUNBOOK.md`
- `docs/FESTIVAL-DATA-MIGRATION.md`